



**Waiting for everyone
to join**

**TU257
Information Systems
Marisa Llorens**

Féidearthachtaí as Cuimse
Infinite Possibilities

Week 4 Keys and Constraints

Information Systems

TU257



Agenda

- Introduction to Logical Design
 - Identifiers/keys
 - Constrains
 - Referential actions
 - Data anomalies
 - Introduction to normalisation

Data Models

- Specify concepts that are used to describe the database. These can be categorised as:
 - *High Level (Conceptual)*
 - *Representational (Logical)*
 - *Low Level (Physical)*

Database Terminology

- **What is a Table/Relation?**

- Single valued cells (groups and arrays are not allowed to be values).
- All entries in any attribute are of the same kind.
- Each column must have a unique name.
 - **Note:** The order of the columns is not significant.
- No two rows are identical.
 - **Note:** The order of the rows is not significant.

S#	NAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clarke	20	London
S5	Adams	30	Athens

Database Terminology

- **Relation:** Relation (in relational model) is **not** the same as relationship (in ER model)
- **Tuple:** Row
- **Attribute:** A property or characteristic of an entity or relationship type that is of interest to the enterprise.

Relational Model	Programmer	User
Relation	File	Table
Tuple (Row)	Record	Row
Attribute	Field	Column

Database Terminology

- **Correspondence with ER Diagram:**
 - **Relations** (tables) correspond with **entity types**.
 - **Rows** correspond with entity **instances**.
 - **Columns** correspond with **attributes**.

S#	NAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clarke	20	London
S5	Adams	30	Athens

Database Terminology

- **Integrity:**
 - The **accuracy** and **correctness** of data.
 - DBMS **enforce** integrity.
 - A **constraint** is any rule governing static values of attributes that is precise enough so that you can ascertain whether or not it is true.

Identifiers/keys

Identifiers/Keys

- A **key** is one or more attributes (together) that uniquely identifies a tuple or row.
- **Primary Key:** The column or set of columns that uniquely identifies any given row.
 - Would a name be a good primary key?
Would a social insurance number be a good primary key?
The value in the primary key can **never** point to more than one row in the table.

Identifiers/Keys Notation

- A table and its primary key can be described by writing the columns within parentheses, and underlining the primary key:

STUDENTS (StudentID, Name, Address, PhoneNumber, DOB)

Identifiers/Keys Types

- **Types of key:**
 - **Single vs. Combination/Composite Key**
 - **Superkeys**
 - **Candidate Keys**
 - Primary Keys vs. Alternative Keys
 - **Foreign Keys**

Identifiers/Keys

- Single

STUDENTS (StudentID, Stu_Name, Address, PhoneNumber)

MODULES (ModuleID, Module_Name)

- Combination Key

STUDENT_MODULE_REGISTRATION (StudentID, ModuleID,
year, semester)

Identifiers/Keys Superkey

- **Superkey:**
 - An attribute or set of attributes that uniquely identify rows in a table.
 - i.e. two distinct rows are always guaranteed to have distinct superkeys.

Example Superkeys

Stud_ID	LastName	specialization	credits	stage	PPS_number
1001	Jones	Data Analytics	20	1	123456
1022	Murphy	Data Analytics	55	4	431312
2012	Ryan	Development	35	3	321231

- Stud_ID
- Stud_ID, LastName
- Stud_ID, LastName, specialization
- PPS_number
- PPS_number, LastName
- PPS_number, stage
-
- Stud_ID, LastName, specialization, credits, stage, PPS_number

Identifiers/Keys Candidate Key

- **Candidate Key:**
 - A minimal superkey
 - i.e. a superkey for which you can say that no proper subset of it is also a superkey.

Example superkeys and candidate keys

- Superkeys:

- Stud_ID
- Stud_ID, LastName
- PPS_number
- PPS_number, LastName
-

- Candidate keys:

- Stud_ID
- PPS_number

Identifiers/Keys PK

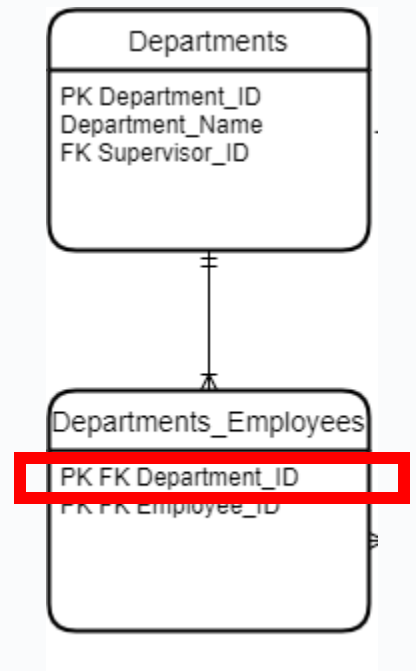
- **Primary Key and alternative keys:**
 - Exactly one candidate key must be chosen as the primary key, and the others are alternative keys.
 - Primary keys and alternative keys are both candidate keys.

Identifiers/Keys FKs

- **Foreign Key:**
 - A foreign key is a reference to a key in another table.
 - A table's primary key is used to establish relationships with other tables.
 - This is achieved by including the primary key of one table as the ***Foreign Key*** of another.

Identifiers/Keys

- The general rule of thumb is to take the primary key of the *one* side of the relationship and include it in the table of the *many* side of the relationship.



Identifiers/Keys

- **Non-prime Attributes:**
 - An attribute that does not occur in any candidate key.

Constraints

Constraint Classification

- **Type (Domain) Constraint:** Define or constrain values that may be assigned to an attribute.
 - e.g. Define a type WEIGHT
 - real numbers.
 - legal values in $[0, 1000]$

Constraint Classification

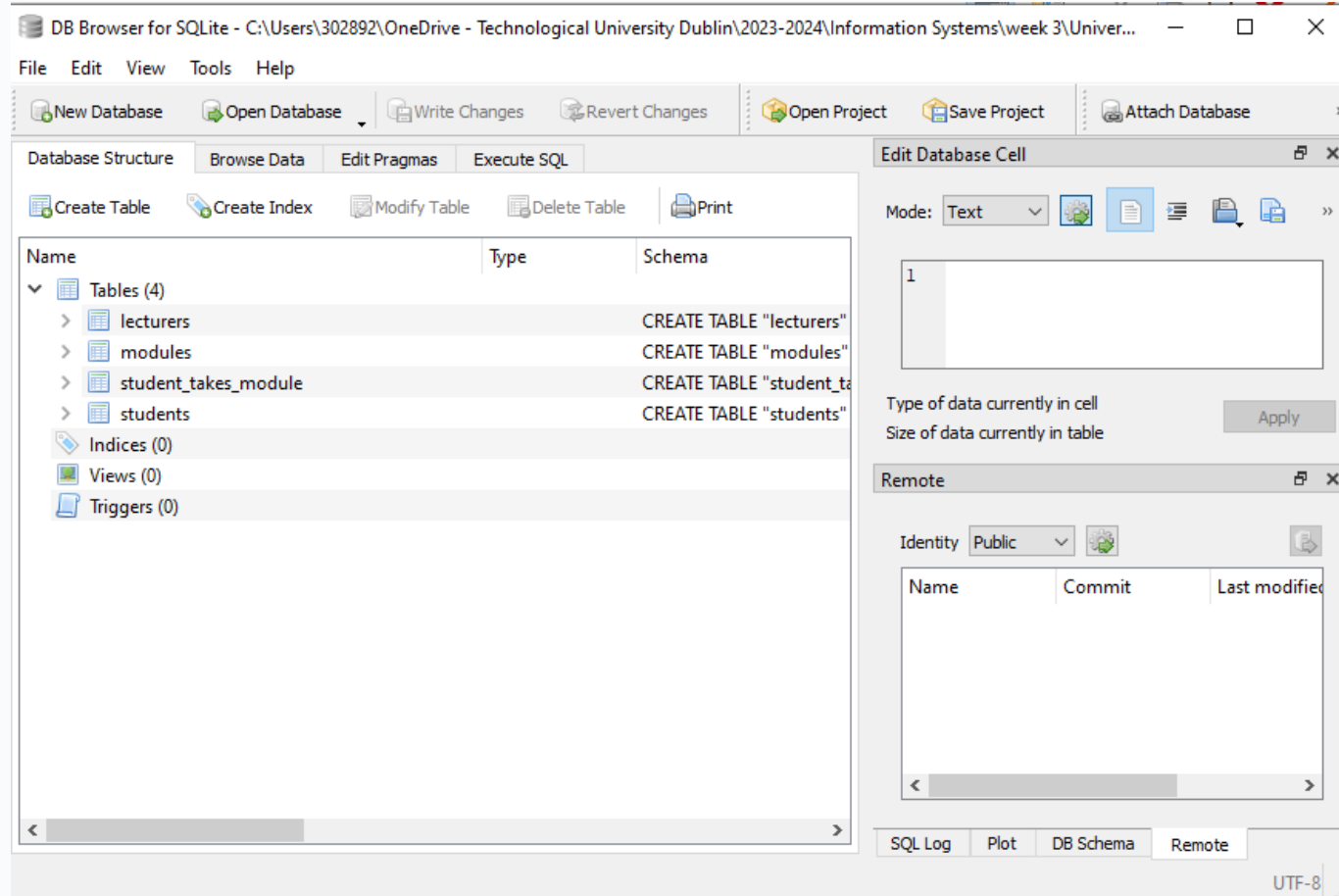
- **Entity Integrity Constraint:** Every row must have a non-null value for the primary key.
 - If it is a composite key, no part of the key may be null.
 - **Null:** Term used to describe a missing value for an attribute in a particular row.
 - It is not the same as zero

Constraint Classification

- **Referential Integrity constraint:**
 - A rule that states that either each foreign key value must match a primary key value in another relation or the foreign key value must be null.



Practical Example University 2



Practice

- Try to add the following data to the table

Student_takes_modules:

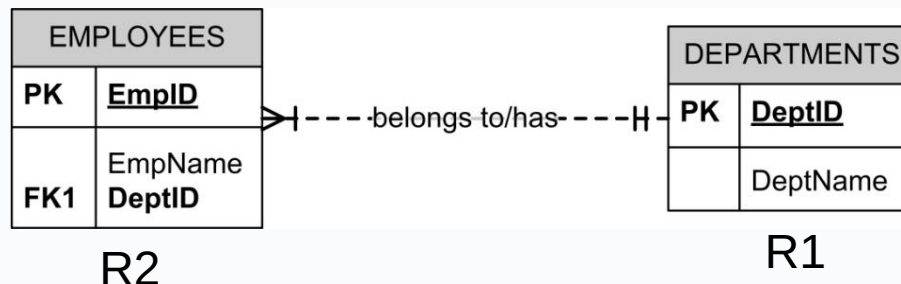
Student_ID	Module_ID
1	103
2	109
1	444
444	105

- What happened?

Referential Actions

Referential Actions ON DELETE

- When you delete a tuple from a table, say **R1**, that is referenced by another table, say **R2**, if the primary key (**K#**) value of **R1** exists in **R2**, there are several choices of referential actions:



Referential Actions ON DELETE

- **ON DELETE CASCADE**

- The corresponding tuples in **R2** will be **deleted** too.

- **ON DELETE RESTRICT**

- The deletion of the tuple from **R1** is **rejected**



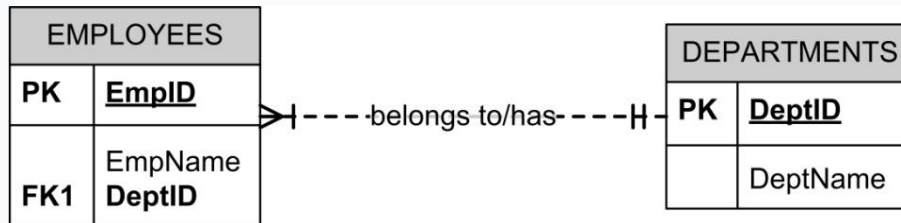
- **ON DELETE NO ACTION**

- The tuple is deleted from **R1**, but no action to **R2**.
- The referential integrity is violated.

Referential Actions ON UPDATE

- When you update a tuple from a table, say **R1**, that is referenced by another table, say **R2**, there are similar choices of referential actions:
 - **ON UPDATE CASCADE**
 - **ON UPDATE RESTRICT**
 - **ON UPDATE NO ACTION**

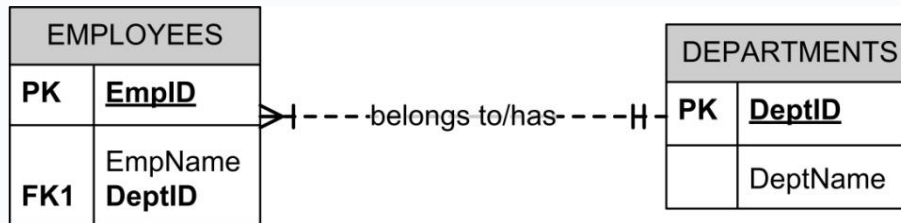
Example: Referential Integrity



EmpID	EmpName	DeptID
123	Sarah O'Connor	A
356	Alice Smith	B

DepID	DepName
A	Web development
B	General Engineering

Example: Referential Integrity ON DELETE CASCADE

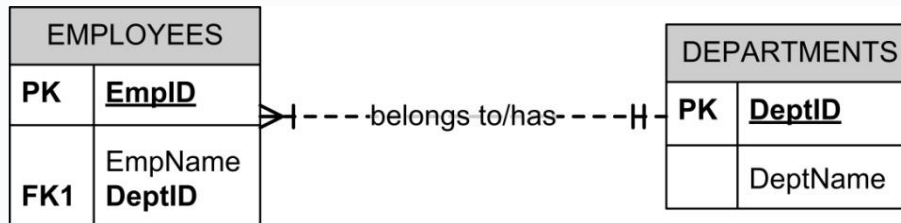


EmpID	EmpName	DeptID
123	Sarah O'Connor	A
356	Alice Smith	B

DepID	DepName
A	Web development
B	General Engineering



Example: Referential Integrity ON DELETE CASCADE

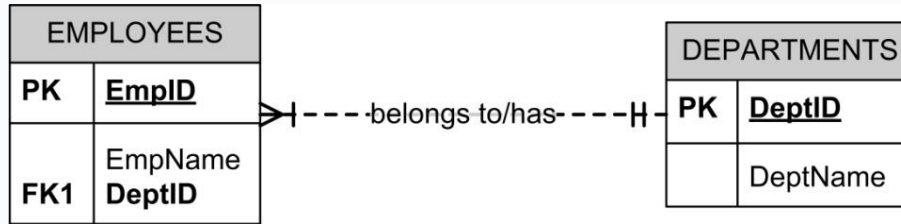


EmpID	EmpName	DeptID
123	Sarah O'Connor	A
356	Alice Smith	B

DepID	DepName
A	Web development
B	General Engineering



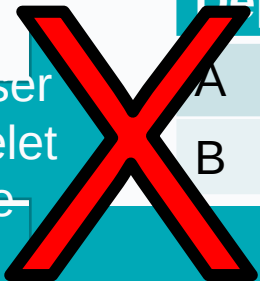
Example: Referential Integrity ON DELETE RESTRICT



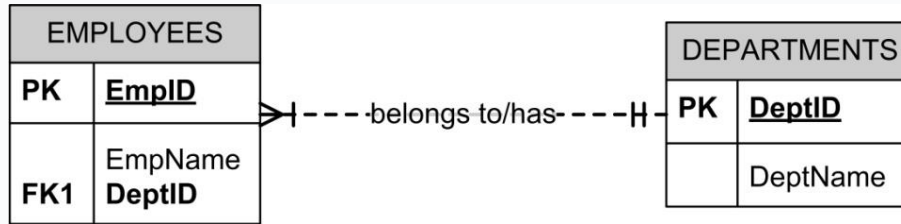
EmpID	EmpName	DeptID
123	Sarah O'Connor	A
356	Alice Smith	B

DepID	DepName
A	Web development
B	General Engineering

User
Delet
e



Example: Referential Integrity ON DELETE NO ACTION



EmpID	EmpName	DeptID
123	Sarah O'Connor	A
356	Alice Smith	B

DepID	DepName
A	Web development
B	General Engineering

The referential integrity is violated



Referential Integrity

ON DELETE NO ACTION

Different DBMS

In MySQL:

- NO ACTION is equivalent to RESTRICT.
- Omitting the clause is the same as specifying RESTRICT or NO ACTION.

In SQLite:

If an action is not explicitly specified, it defaults to "NO ACTION".

Practical Example University 2

- Add the following constraint:

FOREIGN KEY("StudentID") REFERENCES

"students"("StudentID") ON DELETE CASCADE,

- Delete student 1
- Look at the Student_takes_module table to see the effect

Practical Example University 2

StudentID	ModuleID
Filter	Filter
1	101
1	102
2	101
3	103
3	104
4	102
5	101
5	105
6	106
7	107
8	108
9	109
10	110

1 - 14 of 23

StudentID	ModuleID
filter	Filter
2	101
3	103
3	104
4	102
5	101
5	105
6	106
7	107
8	108
9	109
10	110
11	111
12	112

1 - 14 of 21

Normalisation

Normalisation

- The process of designing a database in the **most efficient** way possible.
- A technique for designing relational database tables to minimise **duplication of information** and, in doing so, to safeguard the database against certain types of **logical inconsistency**.

Normalisation

- More highly normalised tables greatly **simplify development** and **maintenance** of the database and **contribute** to its **extensibility**.

Normalisation

- Properly executed, the normalisation process **eliminates uncontrolled data redundancies**,
 - thus **eliminating** the **data anomalies**, and
 - the **data integrity problems** that are produced by such redundancies.

Normalisation

- Normalisation **does not** fully **eliminate data redundancy**.

Instead, it produces the **carefully controlled redundancy** that lets you properly link database tables.

Definitions relevant to normalisation

- Data anomalies
- Repeating groups
- Dependencies
 - Functional and fully functional
 - Partial
 - Transitive

Data Anomalies

- The most common **anomalies** considered when data redundancy exists are:
 - **Update** anomalies
 - **Insert** anomalies
 - **Deletion** anomalies

All these can easily be **avoided** through **data normalisation**.

Example: Data Anomalies

The following relation contains staff and department details.

StaffNo	Job	Dept	DeptName	City
SL10	Salesman	10	Sales	Sligo
SA51	Manager	20	Accounts	Dublin
DS40	Clerk	20	Accounts	Dublin
OS45	Clerk	30	Operations	Dublin



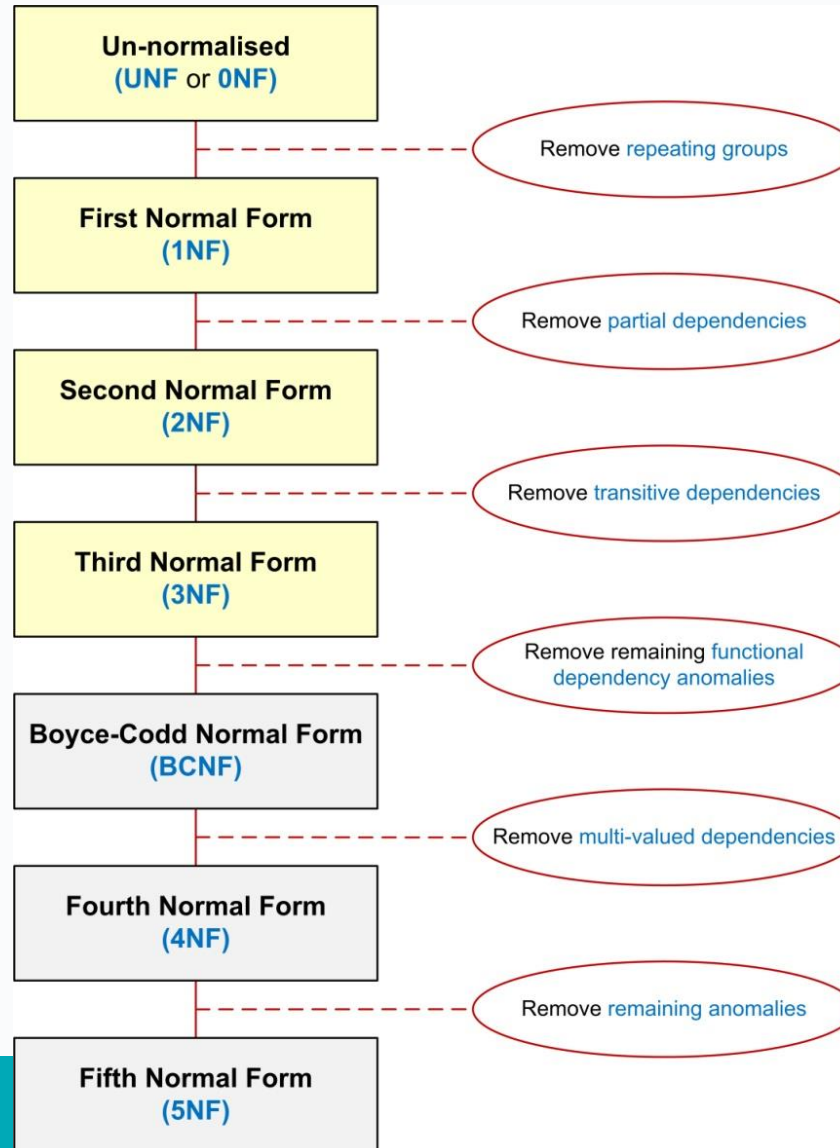
Such '**redundancy**'
could lead to the
following '**anomalies**'

Insert Anomaly: You could insert a new staff member and add incorrect details for the Department.

Update Anomaly: You could change the name of the 'Dept' that SA51 ('StaffNo') works in without simultaneously changing the 'Dept' that DS40 ('StaffNo') works in.

Deletion Anomaly: By removing employee SL10 ('StaffNo') you have removed all information pertaining to the 'Sales Dept' ('Dept', 'DeptName' & 'City').

Stages of Normalisation



Recap: Purpose of Normalisation

- To **avoid redundancy** by storing each '**fact**' within the database only once.
- To put data into a form that **conforms** to **relational principles**
 - single valued attributes,
 - each relation represents one entity, i.e. **No Repeating Groups**

Recap: Purpose of Normalisation

- To put the data into a form that is more able to **accurately accommodate change**.
- To **avoid** certain updating 'anomalies'.
- To **facilitate** the **enforcement** of **data constraints**.

Normalisation: Relational Model

In order to comply with the relational model it is necessary to

1. Remove **repeating groups**, and
2. Avoid **redundancy** and **data anomalies** by removing **partial** and **transitive functional dependencies**.

All attributes in a table must be '**atomic**', and **solely dependant** fully upon the **primary key** of that table.

Thanks

- Edmund Nevin for a earlier versions of these lecture notes

Lab 4

A university library needs a new DBMS. The system needs to handle users borrowing and returning books and keep track of the overall status of the library. The library is located across several sites and all students can borrow up to 10 books at a time from any of the sites. For some popular books, the library keeps multiple copies across different sites to facilitate students' access. Details of site manager, authors and publishers are also kept.

Lab 4

Extra requirements:

- Each site has one manager and managers can only manage one site.
- Assume books only have one author (for now).

Pay attention

- A copy of the book is different to the generic book
- Decide what information is needed in the database and what can be calculated at a later stage



**Break
Back at 8pm
for Lab session**

**TU257
Information Systems
Marisa Llorens**